

Natural Language Processing a.y. 2024/2025

Middle course assignment: *Document Similarity Search*

This assignment aims to build a search engine that returns the most similar documents for a given query. This project uses embeddings to represent documents and similarity measures to retrieve the closest matches from a corpus. This can be applied to any domain with text data, such as news articles, scientific papers, or legal documents.

Steps to Implement

1. Data Collection and Preprocessing

- **Dataset Selection:** Select or download a dataset of documents, such as:
 - ⇒ News articles (e.g., Reuters, 20 Newsgroups dataset)
 - ⇒ Academic papers (e.g., arXiv abstracts or PubMed abstracts)
 - ⇒ E-books (e.g., Project Gutenberg texts in a specific category)
- **Data Cleaning:** Preprocess the text by:
 - ⇒ Lowercasing, removing punctuation, and cleaning unwanted characters.
 - ⇒ Tokenizing the text (breaking it into words or subwords).
 - ⇒ Removing stop words (optional, depending on your approach).
 - ⇒ Stemming or lemmatization to reduce words to their root forms.

2. Document Representation

- **Using Word Embeddings:**
 - ⇒ **Word2Vec:** Train embeddings on your corpus using CBOW to convert words into fixed-size vectors.
 - ⇒ **Average Word Embedding for Each Document:** Create a single vector representation for each document by averaging the embeddings of the words within the document.

3. Similarity Calculation

- **Cosine Similarity:** Measure the cosine similarity between the query vector and each document vector. Cosine similarity calculates the angle between two vectors and provides a similarity score between -1 and 1, where 1 means the vectors are identical in direction.

4. Building the Search Engine

- **Indexing:** Store document embeddings in a way that allows for efficient similarity retrieval. This can be done with a simple list of vectors or a more optimized structure like a k-d tree for larger datasets.
- **Query Processing:**
 - ⇒ Preprocess the query similarly to your documents.
 - ⇒ Convert the query to a vector using the same method as for documents (averaging word embeddings).
- **Similarity Scoring:** Compute similarity scores between the query vector and each document vector, then sort and retrieve the top N most similar documents.

5. Evaluation and Refinement

- **Relevance Assessment:** Check the relevance of the retrieved documents based on the query. You can manually inspect or set up test queries and expected results.
- **Evaluation Metrics:** Use precision, recall, and F1 score if you have a labeled test set with relevance scores.

Optional Extensions

- **Try dimensionality reduction techniques** (e.g., PCA, t-SNE) for visualization.
- **Use Document Embeddings:**

- **Doc2Vec:** Train a Doc2Vec model on your corpus, directly creating embeddings for entire documents. This approach captures the context of each document better than averaging word embeddings.
- **TF-IDF Weighted Word Embeddings:** Instead of simply averaging word embeddings, use a weighted average based on TF-IDF scores, giving more weight to important words in each document.
- **Clustering and Topic Modeling:** Cluster documents into topics using K-Means, to see if clustering helps improve the relevance of search results.

Tools and Libraries

- **Python Libraries:** Use NLTK for text preprocessing, and Scikit-learn for similarity measures and evaluation.
- **PyTorch:** Experiment with training custom embeddings or use neural-based embeddings.